Université des Sciences et de la Technologie d'Oran – Mohamed Boudiaf
Faculté des Mathématiques et Informatique
Département d'Informatique
Master I – Bases de données avancées

# TP révision

*"Révision SQL – Base EL_SAHA – الصحة"*

**Objectifs**

Installer la base EL_SAHA, charger son schéma, explorer le dictionnaire Oracle, puis réviser SQL : requêtes, jointures, vues, agrégations, sous-requêtes, mises à jour et transactions.

| | |
|---|---|
| **Base** | `EL_SAHA` – الصحة |
| **SGBD** | Oracle Database 10g Release 2 |
| **Compte de travail** | `ELSAHA_ADMIN` (`SYSTEM` seulement pour l'installation) |
| **Scripts** | `01_creation_EL_SAHA_v2.sql` et `02_peupler_EL_SAHA_v2.sql` |
| **Document annexe** | `Fiche_MCD_EL_SAHA` : MCD, modèle relationnel, règles de gestion |
| **Questions** | **OBL** : obligatoires ; **OPT** : facultatives |

**Attention**

Toutes les données sont fictives. Il est interdit d'utiliser des données réelles de patients.

**Outil de travail : SQL Developer**

**SQL Developer est l'outil recommandé** (SQL*Plus reste accepté : les scripts fonctionnent dans les deux). Chaque étudiant l'installe et choisit sa version.

- **Installation** : télécharger SQL Developer sur le site d'Oracle ; l'archive Windows « avec JDK inclus » se décompresse et se lance avec `sqldeveloper.exe`.
- **Version** : libre, mais elle doit pouvoir se connecter à Oracle 10g ; en cas d'échec, essayer une autre version.
- **Connexion** : utilisateur `ELSAHA_ADMIN`, hôte `localhost`, port `1521`, SID `EL_SAHA` ; cliquer sur « Tester ».
- **Exécuter** : Ctrl+Entrée pour l'instruction sous le curseur ; F5 pour tout le script (scripts `01` et `02`, `@`, `SPOOL`, blocs PL/SQL).
- **Résultats** : « Résultats de la requête » pour les tables, « Sortie du script » pour les messages et erreurs.

Erreurs de connexion : `ORA-12541` listener arrêté, `ORA-12514` service inconnu, `ORA-01017` mot de passe, `ORA-28000` compte verrouillé.

## 1 Vocabulaire Oracle

- **Base** : fichiers physiques ; **instance** : mémoire et processus qui y donnent accès.
- **Service** et **SID** : noms utilisés pour se connecter (ici `EL_SAHA`).
- **Utilisateur** : compte ; **schéma** : ensemble des objets de ce compte (tables, vues, séquences, …).

- **Tablespace** : conteneur logique de stockage associé à des fichiers de données.

## 2 Créer la base avec DBCA

1. Lancer **Database Configuration Assistant** en administrateur, puis **Créer une base de données**.
2. Modèle **General Purpose / Transaction Processing**.
3. Nom global et SID : `EL_SAHA`.
4. Mots de passe de `SYS` et `SYSTEM` : à retenir, à ne jamais publier.
5. Stockage sur système de fichiers ; noter le répertoire proposé.
6. Étape *Initialization Parameters*, onglet **Memory** : **Typical**, **Multipurpose**, pourcentage réglé pour environ 1 Go au total (25 % pour 4 Go de RAM, 12 % pour 8 Go) ; ne jamais dépasser la valeur par défaut.
7. Même étape, onglet **Character Sets** : cocher **Use Unicode (AL32UTF8)** (pas « Use the default ») ; garder `AL16UTF16` pour le jeu national. Ce choix ne se change pas facilement après.
8. Valider, attendre la fin sans interrompre, noter SID, port et répertoire des fichiers, redémarrer si demandé.

**Attention**

Après un échec, ne pas recréer la base : lire d'abord les journaux DBCA et supprimer le SID résiduel.

Vérifier les services `OracleServiceEL_SAHA` et le listener (`services.msc`), puis, dans une console administrateur : `lsnrctl status`, `tnsping EL_SAHA`. Avec `SYSTEM` :

```
SELECT name, open_mode FROM v$database;
SELECT instance_name, status, version FROM v$instance;
SELECT parameter, value FROM nls_database_parameters
WHERE parameter IN ('NLS_CHARACTERSET','NLS_NCHAR_CHARACTERSET');
```

Résultats attendus : `READ WRITE`, `OPEN`, `AL32UTF8`.

## 3 Créer le tablespace et le compte de travail

Avec `SYSTEM`, repérer un emplacement valide (`SELECT file_name FROM dba_data_files;`), adapter le chemin, puis exécuter :

```
CREATE TABLESPACE elsaha_ts
DATAFILE '<CHEMIN_AUTORISE>/elsaha01.dbf'
SIZE 200M AUTOEXTEND ON NEXT 20M MAXSIZE 2G;

CREATE USER elsaha_admin IDENTIFIED BY mot_de_passe_temporaire
DEFAULT TABLESPACE elsaha_ts TEMPORARY TABLESPACE temp
QUOTA UNLIMITED ON elsaha_ts ACCOUNT UNLOCK;

GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE SEQUENCE,
      CREATE PROCEDURE, CREATE TRIGGER, CREATE SYNONYM TO elsaha_admin;
```

Se déconnecter de `SYSTEM` et se connecter avec `ELSAHA_ADMIN` : **tous les TP se font avec ce compte**.

## 4 Installer le schéma et les données

Placer les scripts dans un chemin court, sans accent ni espace, puis les exécuter avec F5 dans SQL Developer, **dans cet ordre** :

```
SPOOL installation_elsaha.txt
@01_creation_EL_SAHA_v2.sql
@02_peupler_EL_SAHA_v2.sql
SPOOL OFF
```

**Attention**

Le script 01 supprime et recrée les objets du compte : une nouvelle exécution efface les données.

Contrôler : 18 tables, 16 séquences, objets au statut `VALID`, et les trois contrôles finaux du script 02 sans aucune ligne.

```
SELECT COUNT(*) FROM user_tables;
SELECT COUNT(*) FROM user_sequences;
SELECT object_type, status, COUNT(*) FROM user_objects
GROUP BY object_type, status ORDER BY object_type;
```

# 5 Modèle de données

Lire la fiche annexe `Fiche_MCD_EL_SAHA` avant les exercices : MCD, modèle relationnel, dictionnaire des relations et règles de gestion. Le MCD est une base de travail : vous pouvez l'enrichir (attribut, contrainte d'intégrité, index secondaire) en respectant les consignes de cette fiche.

# 6 Dictionnaire de données Oracle

Les vues `USER_*` décrivent les objets du compte, `ALL_*` ceux qui sont accessibles, `DBA_*` tous les objets (privilèges requis).

```
SELECT table_name FROM user_tables ORDER BY table_name;

SELECT table_name, column_id, column_name, data_type, nullable
FROM user_tab_columns ORDER BY table_name, column_id;

SELECT table_name, constraint_name, constraint_type
FROM user_constraints ORDER BY table_name, constraint_type;

SELECT c.table_name, cc.column_name, c.r_constraint_name
FROM user_constraints c
JOIN user_cons_columns cc ON cc.constraint_name = c.constraint_name
WHERE c.constraint_type = 'R' ORDER BY c.table_name;

SELECT index_name, table_name, uniqueness FROM user_indexes ORDER BY table_name;
SELECT sequence_name, last_number FROM user_sequences;
```

**Travail demandé**

**[OBL]** Enregistrer avec `SPOOL` un dictionnaire des tables : colonnes, types, possibilité de valeur nulle et contraintes.

# 7 Exercices de révision SQL

Les questions **OBL** sont indispensables : elles couvrent la révision, créent les vues réutilisées ensuite et modifient la base de façon contrôlée. Les questions **OPT** sont facultatives. Respecter l'ordre des parties.

## 7.1 A. Requêtes simples

1. **[OBL]** Afficher l'identifiant, le nom, le prénom et la ville des patients, triés par nom puis par prénom.
2. **[OBL]** Afficher les médicaments dont le stock est inférieur ou égal au seuil d'alerte.
3. **[OBL]** Afficher les rendez-vous planifiés ou confirmés entre deux dates données.
4. **[OBL]** Afficher les patients dont le nom contient la chaîne « ben », sans tenir compte de la casse.
5. **[OPT]** Calculer l'âge approximatif des patients à la date courante avec `MONTHS_BETWEEN`.
6. **[OPT]** Classer les médicaments en « stock critique » (stock inférieur ou égal à la moitié du seuil), « stock faible » (inférieur ou égal au seuil) et « stock normal » avec `CASE`.

## 7.2 B. Jointures

Utiliser la syntaxe `JOIN ... ON`.

7. **[OBL]** Afficher les médecins avec leur identité, leur spécialité, leur service et leur clinique.
8. **[OBL]** Afficher chaque rendez-vous avec le patient, le médecin, le service du médecin, la date et le statut.
9. **[OBL]** Afficher les hospitalisations avec le patient, la chambre, le service et la clinique.
10. **[OBL]** Afficher, pour chaque consultation, le patient, le médecin, son service et sa clinique, avec les identifiants de ces quatre éléments. **Conserver cette requête** : elle sert à la partie C.
11. **[OBL]** Afficher tous les médecins, y compris ceux qui n'ont réalisé aucune consultation.
12. **[OPT]** Afficher le contenu des ordonnances : patient, date, médicament, posologie et quantité.
13. **[OPT]** Afficher les services avec leur clinique et le nom du médecin chef.

## 7.3 C. Les vues

Une *vue* enregistre une requête sous un nom ; elle ne stocke aucune donnée. Les vues créées ici sont réutilisées dans les parties D et G : respecter l'ordre.

14. **[OBL]** Transformer la requête de l'exercice 10 en vue `V_CONSULTATIONS_DETAIL` (`CREATE OR REPLACE VIEW`), puis l'interroger avec `SELECT * FROM`.
15. **[OBL]** À l'aide de la vue, afficher les consultations d'urgence avec le patient, le médecin et la clinique. Comparer avec la requête qu'il aurait fallu écrire sur les tables.
16. **[OBL]** Créer `V_HOSPITALISATIONS_DETAIL` à partir de l'exercice 9, en gardant les identifiants du patient, du service et de la clinique ainsi que le statut du séjour.
17. **[OBL]** Créer `V_FACTURES_DETAIL` : pour chaque facture, le type de source (consultation ou hospitalisation), le patient, le montant net après remise, le total payé et le reste à payer. *Indication* : le patient se retrouve par la consultation ou par l'hospitalisation facturée (`LEFT JOIN` puis `NVL`) ; les paiements se totalisent dans une sous-requête.
18. **[OBL]** Montrer qu'une vue ne contient pas de données : augmenter les honoraires d'une consultation, relire cette consultation dans la vue, puis annuler avec `ROLLBACK`.
19. **[OBL]** Créer `V_PATIENT_ACCUEIL` (identité, sexe, ville, téléphone ; sans antécédents, allergies ni numéro de sécurité sociale) et expliquer en une phrase pourquoi une vue protège les données de santé.
20. **[OPT]** Interroger `USER_VIEWS` et `USER_TAB_COLUMNS` pour afficher le texte de la vue et la liste de ses colonnes.
21. **[OPT]** Créer `V_STOCK_ALERTE` : médicaments sous le seuil d'alerte, avec la colonne `niveau` de l'exercice 6.
22. **[OPT]** Modifier le téléphone d'un patient à travers `V_PATIENT_ACCUEIL`, puis tenter de modifier le nom du patient à travers `V_CONSULTATIONS_DETAIL`. Expliquer l'erreur obtenue (`ORA-01779`), puis `ROLLBACK`.

## 7.4 D. Agrégations

Utiliser les vues de la partie C quand l'énoncé le précise.

23. **[OBL]** Compter le nombre de patients par ville.
24. **[OBL]** Calculer, pour chaque médecin, le nombre de consultations et le montant moyen des honoraires (utiliser `V_CONSULTATIONS_DETAIL`), en ne gardant que les médecins ayant réalisé au moins cinq consultations.
25. **[OBL]** Calculer le chiffre d'affaires net (après remise) facturé par clinique pour les consultations (utiliser la vue et la table `FACTURE`).
26. **[OBL]** Afficher les factures dont le reste à payer dépasse 5 000 DA (utiliser `V_FACTURES_DETAIL`).
27. **[OPT]** Compter le nombre de médecins par clinique et spécialité.
28. **[OPT]** Afficher les cinq médicaments les plus prescrits (nombre d'ordonnances).
29. **[OPT]** Calculer le taux d'occupation observé par service : hospitalisations en cours rapportées à la capacité en lits (utiliser `V_HOSPITALISATIONS_DETAIL`).

### 7.5 E. Sous-requêtes et opérations ensemblistes

30. **[OBL]** Afficher les patients ayant au moins une consultation avec `EXISTS`.
31. **[OBL]** Afficher les patients n'ayant jamais été hospitalisés avec `NOT EXISTS`.
32. **[OBL]** Afficher les médecins dont le salaire est supérieur au salaire moyen.
33. **[OPT]** Afficher les consultations dont les honoraires sont supérieurs à la moyenne de leur type.
34. **[OPT]** Avec `INTERSECT` puis `MINUS`, afficher les patients ayant une consultation et une hospitalisation, puis ceux qui ont une consultation sans hospitalisation.

### 7.6 F. Manipulation et transactions

Travailler sur des lignes d'essai identifiables. **À la fin de la partie F, la base doit être revenue à son état initial** (50 patients, 100 rendez-vous) : les TP suivants en dépendent. Le patient n'a plus de table `PERSONNE` : une seule insertion suffit.

35. **[OBL]** Insérer un patient d'essai (utiliser `SEQ_PATIENT.NEXTVAL`).
36. **[OBL]** Ajouter un rendez-vous pour ce patient avec un médecin existant en respectant toutes les clés étrangères (`SEQ_RDV`, `SEQ_PATIENT.CURRVAL`).
37. **[OBL]** Modifier le statut de ce rendez-vous en `CONFIRME`.
38. **[OBL]** Créer un `SAVEPOINT`, modifier le stock d'un médicament, puis annuler uniquement cette modification.
39. **[OBL]** Supprimer le rendez-vous d'essai puis le patient d'essai (expliquer pourquoi dans cet ordre), puis valider avec `COMMIT`.
40. **[OPT]** Provoquer trois erreurs d'intégrité et expliquer chacune : (a) un rendez-vous pour un patient inexistant ; (b) une quantité négative dans `PRESCRIPTION` ; (c) la suppression d'un service qui possède des médecins.

### 7.7 G. Index et synthèse

41. **[OPT]** Créer un index secondaire utile (par exemple sur `RENDEZ_VOUS(STATUT)`) et vérifier sa présence dans `USER_INDEXES`.
42. **[OPT]** Produire une requête de synthèse donnant, par clinique, le nombre de services, de médecins, de consultations, d'hospitalisations et le total net facturé (utiliser les vues).

## 8 Travail à rendre

- `installation_elsaha.txt` (SPOOL) et le dictionnaire des tables ;
- `revision_nom_binome.sql` : requêtes numérotées et commentées, avec la création des vues ;
- le résultat des questions **OBL**, y compris la vérification finale ci-dessous ;
- trois intérêts d'une vue, en une phrase chacun ;
- les erreurs rencontrées et leur correction (sans mot de passe).

## 9 Vérification finale

La base doit être revenue à son état initial :

```
  SELECT 'CLINIQUE' table_nom, COUNT(*) nombre FROM clinique
UNION ALL SELECT 'PATIENT' table_nom, COUNT(*) nombre FROM patient
UNION ALL SELECT 'SERVICE' table_nom, COUNT(*) nombre FROM service
UNION ALL SELECT 'MEDECIN' table_nom, COUNT(*) nombre FROM medecin
UNION ALL SELECT 'CHAMBRE' table_nom, COUNT(*) nombre FROM chambre
UNION ALL SELECT 'RENDEZ_VOUS' table_nom, COUNT(*) nombre FROM rendez_vous
UNION ALL SELECT 'CONSULTATION' table_nom, COUNT(*) nombre FROM consultation
UNION ALL SELECT 'ORDONNANCE' table_nom, COUNT(*) nombre FROM ordonnance
UNION ALL SELECT 'MEDICAMENT' table_nom, COUNT(*) nombre FROM medicament
UNION ALL SELECT 'PRESCRIPTION' table_nom, COUNT(*) nombre FROM prescription
UNION ALL SELECT 'ACTE' table_nom, COUNT(*) nombre FROM acte
UNION ALL SELECT 'ACTE_CONSULTATION' table_nom, COUNT(*) nombre FROM acte_consultation
```

```sql
UNION ALL SELECT 'ANALYSE' table_nom, COUNT(*) nombre FROM analyse
UNION ALL SELECT 'DEMANDE_ANALYSE' table_nom, COUNT(*) nombre FROM demande_analyse
UNION ALL SELECT 'HOSPITALISATION' table_nom, COUNT(*) nombre FROM hospitalisation
UNION ALL SELECT 'FACTURE' table_nom, COUNT(*) nombre FROM facture
UNION ALL SELECT 'PAIEMENT' table_nom, COUNT(*) nombre FROM paiement;
```

| | | | | | |
|---|---|---|---|---|---|
| CLINIQUE | 4 | PATIENT | 50 | SERVICE | 12 |
| MEDECIN | 15 | CHAMBRE | 24 | RENDEZ_VOUS | 100 |
| CONSULTATION | 70 | ORDONNANCE | 50 | MEDICAMENT | 25 |
| PRESCRIPTION | 109 | ACTE | 15 | ACTE_CONSULTATION | 93 |
| ANALYSE | 12 | DEMANDE_ANALYSE | 60 | HOSPITALISATION | 20 |
| FACTURE | 80 | PAIEMENT | 62 | | |